



Interrupt-Based Bootloader Design For Embedded System Updates

A bootloader helps to update a system's software without a dedicated programmer. It also solves the purpose of feature enhancement. This article covers how to manage interrupts between the bootloader and application software in case the relocation of the interrupt vector table (IVT) is not allowed



Ashish Madgundi
is working as a senior software engineer at Knorr Bremse Technology Centre India Pvt Ltd, Pune

Millions of microcontrollers are sold every year, and there is always a slight chance of defect in the system, which is observed only when these go live. It is not a good practice to bring back all the systems, fix the defect, and reinstall them. So, there must be some kind of mechanism available to update the system's software in the field itself to fix the defect. A bootloader solves this purpose. The primary function of the bootloader is to update the system's software without a dedicated programmer.

The embedded bootloaders come in different sizes to communicate over different protocols like CAN, I2C, UART, SPI, etc. When we talk about a bootloader, it is assumed that the system contains at least two images existing in the same microcontroller. One image is for the bootloader to update the software, if required, and the other is for the application, which has the system functionality. Additionally, each software must have a branch code to jump from one to another.

The bootloader also solves the purpose

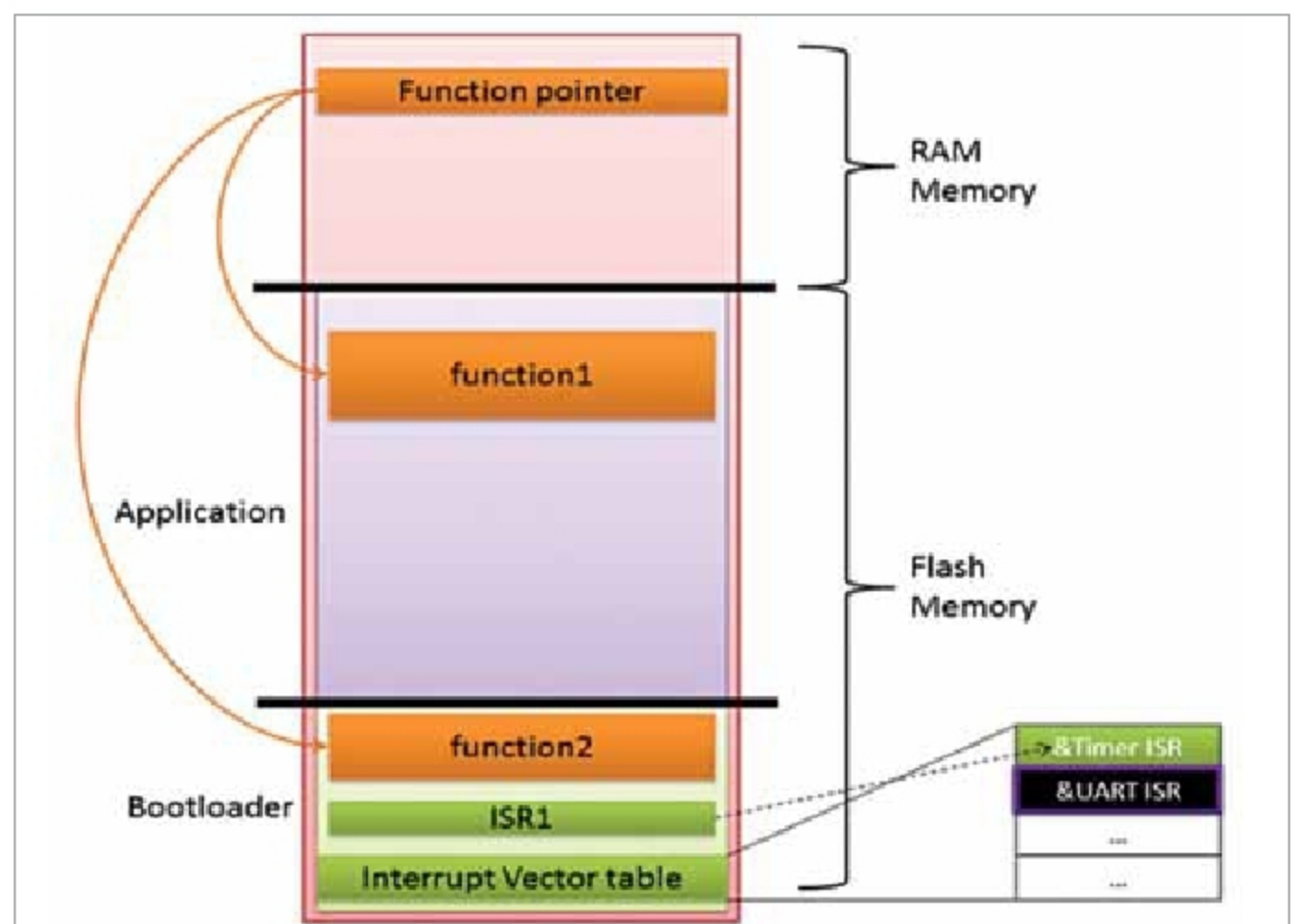


Fig. 1: Memory map of controller